

ACTIVIDAD 1

CONCEPTOS FUNDAMENTALES DE SERVICIOS Y PROCESOS

NOMBRE Y APELLIDOS: Juan Gabriel Galarza Claros

EJERCICIO 1: Responde a las siguientes preguntas
a) ¿Qué es un servicio? b) ¿Qué es un proceso? c) ¿Qué es la programación concurrente? ¿Y la programación multihilo? ¿Son lo mismo?
RESPUESTAS: a) Conjunto de actividades o acciones diseñadas para satisfacer una necesidad o un deseo del consumidor. b) Secuencia de pasos o actividades ordenadas que transforman unos elementos de entrada en un resultado o objetivo final. c) La programación concurrente es la capacidad de un sistema para gestionar múltiples tareas que avanzan en periodos de tiempo solapados; La programación multihilo es una forma concreta de lograr esa concurrencia utilizando múltiples hilos de ejecución dentro de un mismo proceso. No son exactamente lo mismo, sino que la programación multihilo es una herramienta (o un modelo) para implementar la concurrencia
REFERENCIAS: https://es.wikipedia.org/wiki/Servicio_(econom%C3%ADa) https://dpej.rae.es/lema/servicio https://economipedia.com/definiciones/proceso.html https://www.it.uc3m.es/pbasanta/asng/course_notes/threads_es.html https://tecnologia.euroinnova.com/concurrencia

EJERCICIO 2: Responde a las siguientes preguntas
a) ¿Qué es la criptografía? b) ¿Qué son los sockets?
RESPUESTAS: a) La criptografía es ciencia y el arte de proteger la información transformándola mediante algoritmos matemáticos para que solo puedan leerla las personas autorizadas. b) Un socket es un término informático que se refiere principalmente a un conector físico para el procesador en una computadora o a un punto de conexión lógico para la comunicación de datos en una red
REFERENCIAS: https://www.ibm.com/es-es/think/topics/cryptography https://www.sede.fnmt.gob.es/preguntas-frecuentes/otras-preguntas/-/asset_publisher/1RphW9IeUoAH/content/1022-que-es-la-criptografia- https://www.flexygo.com/que-es-un-socket-funcion-y-usos-explicados/ https://www.pccomponentes.com/blog/socket-que-es

EJERCICIO 3: Responde a las siguientes preguntas	
a) Define los conceptos de programa, proceso e hilo, y describe la relación que hay entre ellos.	
RESPUESTA: - Programa • Definición: Es un conjunto de instrucciones escritas en un lenguaje de programación y guardadas en un archivo estático en el disco (por ejemplo, calculadora.exe o app.py). Es una entidad pasiva: no hace nada por sí mismo hasta que se decide ejecutar. - Proceso • Definición: Es una instancia de un programa en ejecución. Cuando el sistema operativo carga un programa en la memoria RAM y le asigna recursos (espacio de dirección de memoria, archivos abiertos, variables), el programa cobra vida y se convierte en un proceso. Es una entidad activa. - Hilo (Thread) • Definición: Es la unidad básica de ejecución dentro de un proceso (también conocido como "proceso ligero"). Representa una secuencia de instrucciones que el procesador puede ejecutar de forma independiente. Un proceso puede contener uno o varios hilos. Relación entre ambos: Programa (El plano): Código estático y pasivo guardado en el disco. Proceso (La construcción): El programa en ejecución, activo y cargado en la memoria RAM. Actúa como el contenedor de recursos. Hilo (El trabajador): La unidad básica de ejecución dentro de un proceso. Un proceso puede tener múltiples hilos. Compartición: Todos los hilos de un mismo proceso comparten su memoria, variables y archivos. Sin embargo, cada hilo mantiene su propio contador, registros y pila de forma individual.	
REFERENCIAS: https://w3.ual.es/~rguirado/so/tema2.pdf https://clasicwebtools.com/articulos/tecnologia/Procesos%20e%20Hilos%20en%20los%20sistemas%20operativos.html	

EJERCICIO 4: Responde a las siguientes preguntas	
a) ¿Qué es la programación concurrente? b) ¿Qué es la programación paralela? c) ¿Qué es la programación distribuida? d) Dame una tabla con las ventajas de la programación concurrente, paralela y distribuida. e) Dame una tabla con los inconvenientes de la programación concurrente, paralela y distribuida.	
RESPUESTAS: - La programación concurrente es un modelo de desarrollo que permite ejecutar múltiples tareas o procesos de forma simultánea o intercalada. - La programación paralela es una técnica de desarrollo de software que divide un problema grande en varias partes más pequeñas para que se ejecuten al mismo tiempo en diferentes procesadores o núcleos de cómputo. - La programación distribuida es un paradigma de desarrollo de software en el que un programa se divide en varias partes que se ejecutan simultáneamente en múltiples ordenadores o nodos conectados a través de una red, trabajando juntos para cumplir un objetivo común.	
REFERENCIAS: https://es.wikipedia.org/wiki/Computaci%C3%B3n_concurrente https://www.ibm.com/es-es/think/topics/parallel-computing	

<https://www.unir.net/revista/ingenieria/computacion-paralela/>
<https://keepcoding.io/blog/que-es-la-programacion-distribuida/>
https://es.wikipedia.org/wiki/Programaci%C3%B3n_distribuida

VENTAJAS	
Paradigma	Principales Ventajas
Concurrente	• Mayor responsividad: La interfaz no se congela ante tareas I/O bloqueantes. • Aprovechamiento de tiempos muertos: Alterna tareas mientras espera respuestas externas (disco, red). • Estructura modular: Facilita desacoplar componentes independientes (ej. productor-consumidor).
Paralela	• Rendimiento y velocidad máxima: Reduce drásticamente el tiempo de ejecución dividiendo carga computacional. • Aprovechamiento del hardware multinúcleo/GPU: Ejecución verdaderamente simultánea en varias CPUs/cores. • Alta eficiencia en tareas pesadas de cómputo: Escalabilidad en problemas masivos de cálculo.
Distribuida	• Alta disponibilidad y tolerancia a fallos: Si un nodo cae, otros asumen la carga sin tumbar el sistema. • Escalabilidad horizontal (scale-out): Permite crecer añadiendo máquinas económicas en clúster. • Compartición de recursos y acceso geográfico: Permite procesar datos cerca de donde se generan o consumen.

INCONVENIENTES	
Paradigma	Principales Inconvenientes
Concurrente	• No determinismo y depuración difícil: Los fallos dependen del <i>timing</i> y rara vez son reproducibles. • Problemas de sincronización: Riesgo constante de <i>deadlocks</i> (interbloqueos), <i>race conditions</i> e inanición. • Sobrecarga de contexto: El cambio de contexto frecuente (<i>context switching</i>) puede penalizar la CPU.
Paralela	• Dependencia directa del hardware: Requiere arquitecturas específicas (multinúcleo, GPUs dedicadas). • Límite de aceleración (Ley de Amdahl): La ganancia se ve acotada por la parte estrictamente secuencial del código. • Complejidad de descomposición: Difícil balancear la carga uniformemente y minimizar contención de memoria/caché.
Distribuida	• Fallas de red y latencia: Las comunicaciones remotas introducen retardos impredecibles y desconexiones. • Consistencia de datos compleja: Obliga a gestionar compromisos teóricos (Teorema CAP) y consistencia eventual. • Operación y monitorización críticas: Requiere infraestructura pesada (orquestación, trazas distribuidas, seguridad de red).